

SE Paper Solution 2025

Part A

1. State any four attributes of good software.

Ans: Attributes of good software are:

Correctness – Performs required functions accurately

Reliability – Operates without frequent failures

Efficiency – Uses minimum resources

Maintainability – Easy to modify and update

2. Define Software Quality Assurance (SQA).

Ans: Software Quality Assurance (SQA) is a systematic process that ensures software development follows defined standards, procedures, and processes to produce high-quality software.

3. Define the term software security.

Ans: Software security refers to protecting software systems from unauthorized access, attacks, misuse, and vulnerabilities to ensure confidentiality, integrity, and availability of data.

4. What is the prime objective of software engineering?

Ans: The prime objective of software engineering is to develop **reliable, efficient, maintainable, and cost-effective software** that satisfies user requirements within given constraints.

5. Why is accuracy an important attribute for data dictionaries?

Ans: Accuracy ensures correct data definitions, avoids ambiguity, maintains consistency across the system, and prevents errors during system analysis, design, and implementation.

6. Explain the term “Risk Analysis”.

Ans: Risk analysis is the process of identifying, evaluating, and prioritizing potential risks that may negatively impact software project cost, schedule, quality, or performance.

SE Paper Solution 2025

7. Differentiate Object-Oriented Analysis (OOA) and Object-Oriented Design (OOD).

OOA	OOD
Focuses on problem analysis	Focuses on solution design
Identifies objects & requirements	Defines classes & architecture
What system should do	How system will be built

8. Distinguish between Process and Method.

Process	Method
Defines overall framework	Defines how tasks are done
High-level approach	Detailed technique
Example: SDLC	Example: OOD, Prototyping

9. Define SRS.

Ans: SRS (Software Requirement Specification) is a formal document that describes functional and non-functional requirements, constraints, and interfaces of a software system.

10. Define Software Engineering Paradigm.

Ans: A software engineering paradigm is a structured approach or philosophy used to develop software, such as Waterfall, Incremental, Spiral, Agile, or Object-Oriented paradigm.

Part B

Q1. What is SDLC? Explain MIS-Oriented SDLC Model.

SDLC (Software Development Life Cycle)

SDLC (Software Development Life Cycle) is a systematic framework used to develop software in a disciplined manner, ensuring quality, cost control, and timely delivery. It defines the complete process from idea conception to software maintenance.

SE Paper Solution 2025

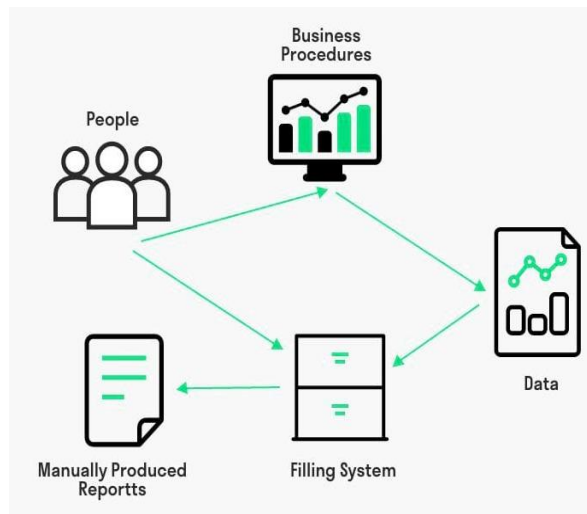
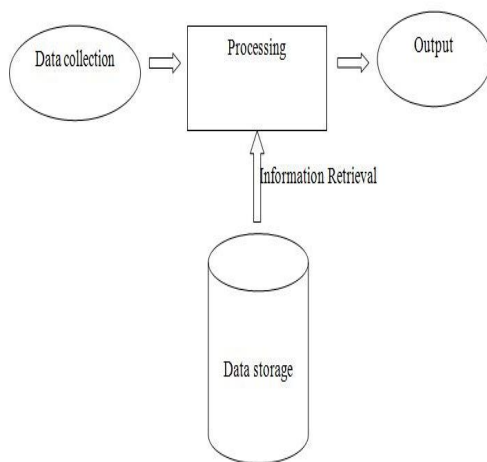
Phases of SDLC

1. Planning
2. Requirement Analysis
3. System Design
4. Coding
5. Testing
6. Implementation
7. Maintenance

MIS-Oriented SDLC Model

The **MIS-oriented SDLC model** is specifically designed for **Management Information Systems**, where the primary objective is to process data and generate meaningful information for decision-making. This model emphasizes structured documentation, data flow, and reporting accuracy.

MIS-SDLC Diagram



Features

- Data-centric
- Strong documentation
- Used in business organizations

SE Paper Solution 2025

Q2. What is UML? How is it beneficial in Object-Oriented Model?

UML (Unified Modeling Language) is a standardized graphical modeling language used to visualize, specify, construct, and document object-oriented software systems. It provides a common visual notation that helps developers, designers, and stakeholders understand system structure and behavior.

UML is highly beneficial in the **object-oriented model** because it directly supports object-oriented concepts such as classes, objects, inheritance, encapsulation, and polymorphism. UML offers various diagrams like **use case diagrams** (to capture user requirements), **class diagrams** (to represent system structure), **sequence diagrams** (to show object interaction), and **state diagrams** (to model object behavior).

Benefits of UML

- Improves system understanding
- Provides visual representation
- Enhances communication
- Supports OOP concepts
- Reduces complexity

UML Usage Diagram

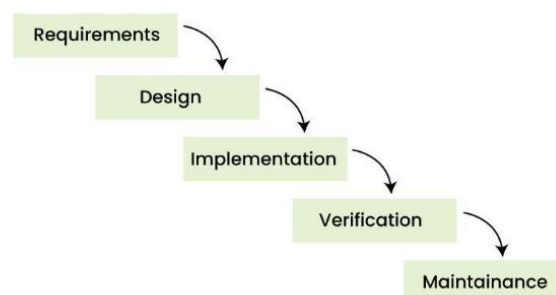
Requirements → UML Diagrams → Design → Implementation

Q3. Explain Waterfall Model and Spiral Model with real-time example.

Waterfall Model

A **linear sequential model** where each phase must be completed before the next begins.

Waterfall Phases



SE Paper Solution 2025

Requirement → Design → Coding → Testing → Deployment → Maintenance

Real-Time Example

- Payroll system
- Library management system(LMS)

Advantages

- Simple and easy to manage
- Clear documentation
- Low cost

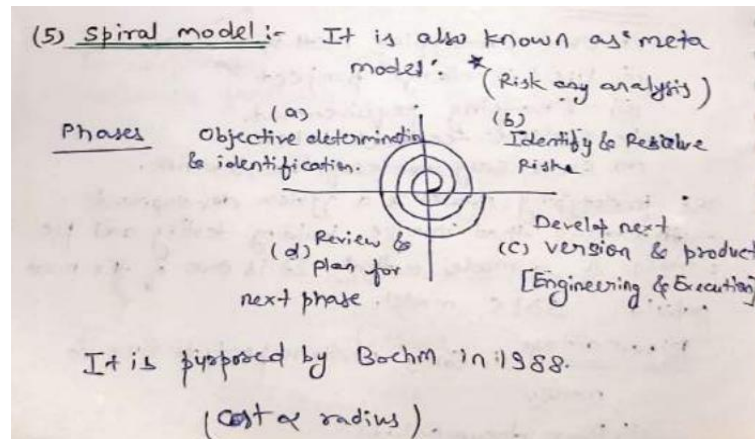
Limitations

- Not flexible
- Changes are difficult
- Low customer involvement

Spiral Model

A risk-driven iterative model combining waterfall and prototyping.

Spiral Diagram



Planning → Risk Analysis → Engineering → Evaluation (Repeated in loops)

Real-Time Example

- Banking software
- Large enterprise systems
- Space research Project

SE Paper Solution 2025

Advantages

- Early risk identification
- Flexible

Limitations

- Access Documentation
- High Budget
- Requirements are unclear and complex

Q4. List and explain techniques to enhance software quality and software reliability.

Software quality refers to how well software meets user requirements and standards, while **software reliability** refers to its ability to perform correctly over time without failure.

Techniques to enhance **software quality** include:

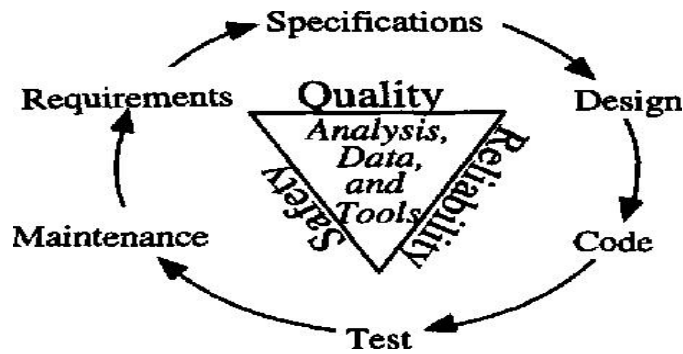
- **Standardized development processes**, ensuring consistency.
- **Code reviews and inspections**, which help detect defects early.
- **Comprehensive testing**, including unit, integration, and system testing.
- **Proper documentation**, which improves understanding and maintenance.



Techniques to enhance **software reliability** include:

- **Fault avoidance**, achieved through good design and coding practices.
- **Fault detection**, using rigorous testing and debugging.
- **Fault tolerance**, by adding error-handling and recovery mechanisms.
- **Reliability testing**, to evaluate system performance under stress.

SE Paper Solution 2025



Quality and reliability are closely related—high-quality software generally leads to higher reliability. By adopting disciplined development practices, continuous testing, and effective error handling, organizations can produce dependable and robust software systems.

Q5. A project size is 200 KLOC. Calculate effort, development time, average staff size, and productivity.

Given: Project size = **200 KLOC**

Assume **Organic mode** of COCOMO

Constants: $a = 2.4$, $b = 1.05$, $c = 2.5$, $d = 0.38$

Effort estimation:

$$\text{Effort} = a \times (\text{KLOC})^b$$

$$\text{Effort} = 2.4 \times (200)^{1.05} \approx \mathbf{566 \text{ Person-Months}}$$

Development time:

$$\text{Time} = c \times (\text{Effort})^d$$

$$\text{Time} = 2.5 \times (566)^{0.38} \approx \mathbf{26 \text{ months}}$$

Average staff size:

$$\text{Staff} = \text{Effort} / \text{Time}$$

$$\text{Staff} = 566 / 26 \approx \mathbf{22 \text{ persons}}$$

Productivity:

$$\text{Productivity} = \text{KLOC} / \text{Effort}$$

$$\text{Productivity} = 200 / 566 \approx \mathbf{0.35 \text{ KLOC per person-month}}$$

This estimation helps project managers plan manpower, schedule, and cost effectively. COCOMO provides a structured approach to project planning and risk reduction.

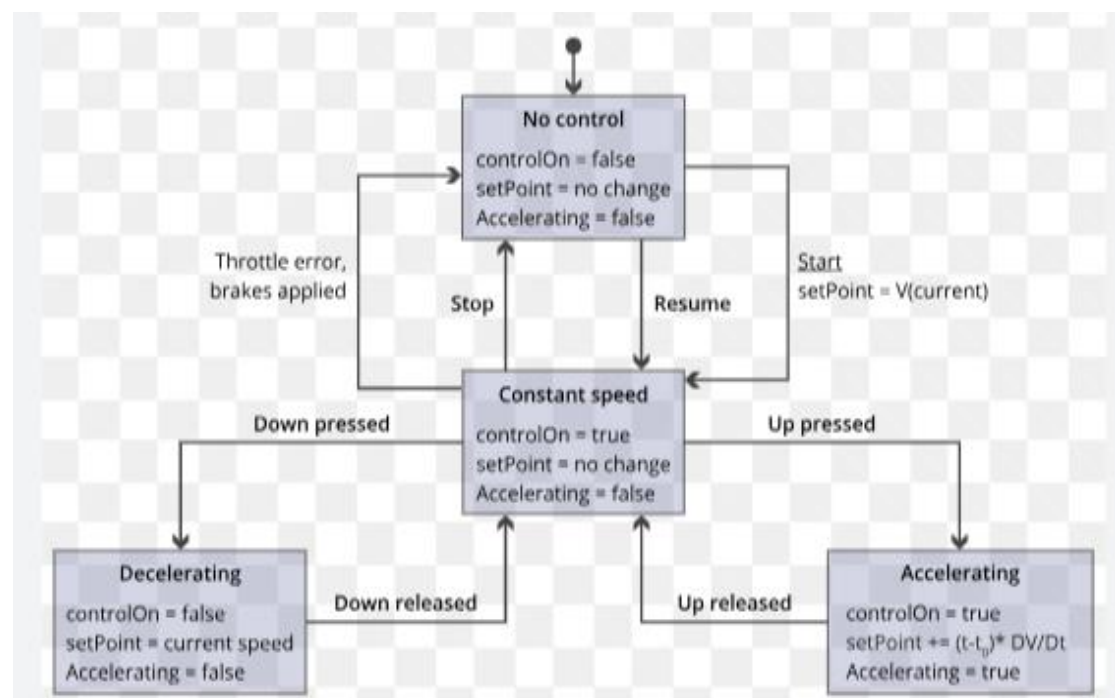
SE Paper Solution 2025

Q6. Explain Finite State Machine (FSM).

A **Finite State Machine (FSM)** is a behavioral model used to represent systems that operate in a finite number of states. At any given time, the system exists in only one state and moves to another state based on input events.

An FSM consists of four main components: **states**, **events (inputs)**, **transitions**, and **actions (outputs)**. When a specific event occurs, the system transitions from one state to another and may perform an action.

FSMs are widely used in **real-time systems, embedded systems, communication protocols, and control systems**. For example, a traffic light system can be modeled using states such as Red, Yellow, and Green.



The main advantages of FSMs are simplicity, clarity, and ease of implementation. However, FSMs may become complex when the number of states increases significantly, leading to the “state explosion” problem.

Q7. What are the approaches of debugging?

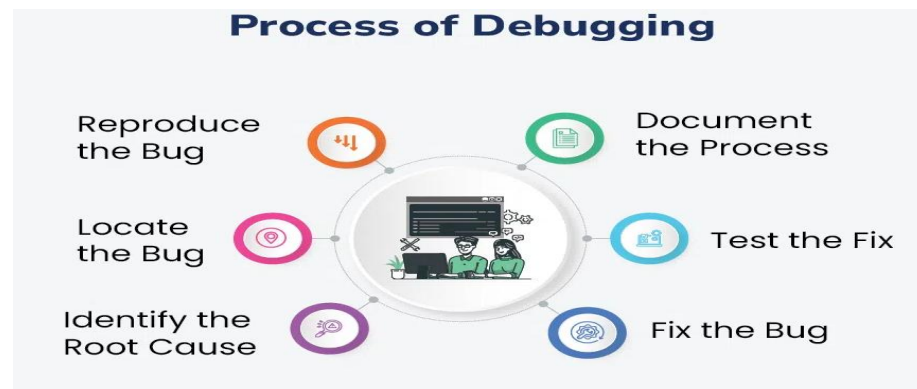
Debugging is the process of identifying, analyzing, and removing errors (bugs) from a software program. It is a critical activity during software testing and maintenance.

SE Paper Solution 2025

The main approaches to debugging are:

- **Brute Force Method** : This approach uses print statements, memory dumps, or debugging tools to trace errors. It is simple but inefficient for large programs.
- **Backtracking** : In this method, the program is traced backward from the point where the error occurred to locate its source. It is effective but time-consuming.
- **Cause Elimination** : Possible causes of the error are identified and systematically eliminated using hypothesis testing.
- **Program Slicing** : This technique isolates the portion of the program related to the error, reducing the search space.

Effective debugging requires logical thinking, good understanding of the program, and proper use of debugging tools.



Part C

Q1. What are the characteristics of a good design? Describe different types of coupling and cohesion. How is design measured?

Characteristics of a Good Software Design

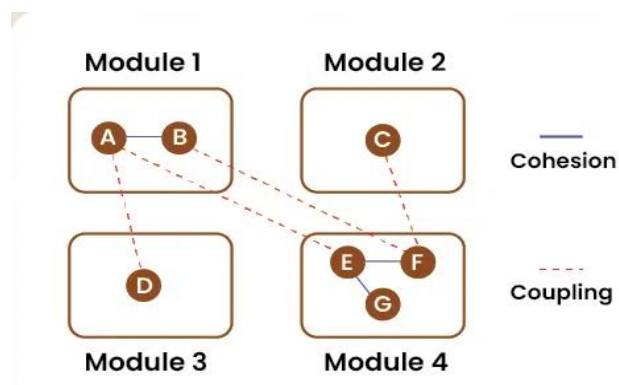
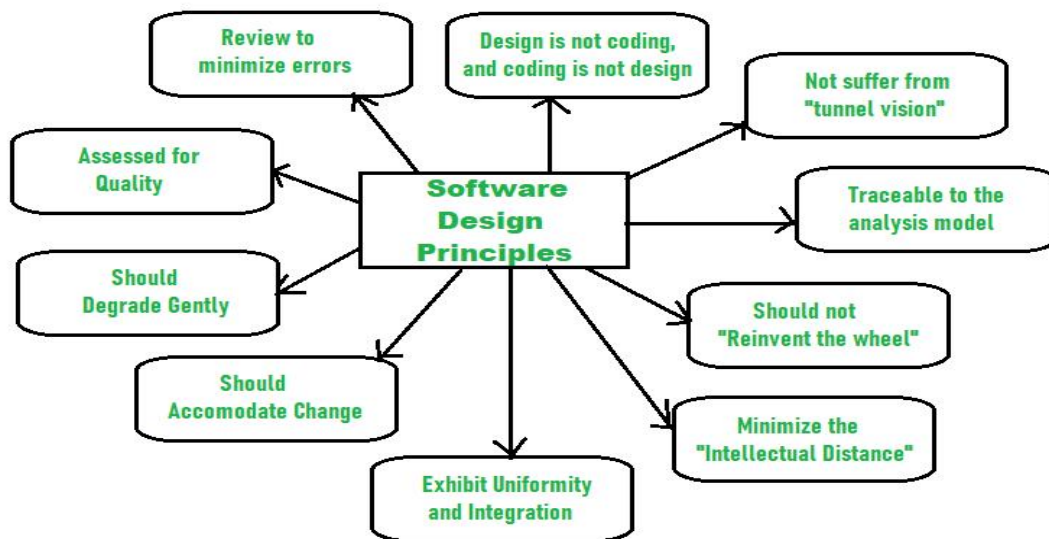
A good software design ensures that the system is efficient, reliable, and easy to maintain.

Key characteristics:

- **Correctness** – Design meets all requirements specified in SRS
- **Modularity** – System is divided into well-defined modules
- **Simplicity** – Easy to understand and implement
- **Efficiency** – Optimal use of time and memory
- **Maintainability** – Easy to modify and extend
- **Reusability** – Components can be reused in other systems

SE Paper Solution 2025

- **Flexibility** – Can accommodate future changes



Cohesion

Cohesion is the measure of how **closely related the elements within a single module are**.

High cohesion is desirable, because it makes the module easy to understand, test, and maintain.

Types of Cohesion (Worst → Best)

1. Coincidental Cohesion (Worst) : Module performs unrelated tasks.
No meaningful relationship among functions

Ex. Module A

- Print
- Sort
- Calculate

SE Paper Solution 2025

2. Logical Cohesion : Related logically but executed based on a control flag

Ex. Module B

├ Read File

├ Write File
└ Open File

3. Temporal Cohesion : Tasks executed at the same time (e.g., initialization)

Ex. Exception handling with the help of catch function.

4. Procedural Cohesion : Tasks follow a specific sequence of execution

Step 1 → Step 2 → Step 3

Ex. Drive access with permission.

5. Communicational Cohesion: Tasks operate on the same data

Module D

├ Read Student Data
└ Update Student Data

6. Sequential Cohesion : Output of one task becomes input of another

Ex. Year-> Branch -> Subject-> Unit

7. Functional Cohesion (Best) : Module performs one single, well-defined function

Coupling

Coupling is the measure of **dependency between modules**.

Low coupling is desirable, as it reduces the impact of changes.

Types of Coupling (Worst → Best)

1. Content Coupling (Worst) : One module directly accesses another module's internal data

2. Common Coupling : Modules share global data

SE Paper Solution 2025

3. External Coupling : dependency on hardware

4. Control Coupling : One module controls the behavior of another using flags. Ex. Data sharing within two module

5. Stamp Coupling : Modules share a composite data structure.Ex. Call by reference or address.

6. Data Coupling (Best) : Modules communicate using simple data parameters. Ex. Call by value or data

Design Measurement

Design quality is measured using:

- Cohesion metrics
- Coupling metrics
- Complexity metrics
- Maintainability index

Q2. Explain the following:

(a) Use Case Diagram

(b) State Chart Diagram

1. Use Case Diagram

A **Use Case Diagram** is a **UML behavioral diagram** that represents the **functional requirements of a system** by showing **interactions between users (actors) and the system**.

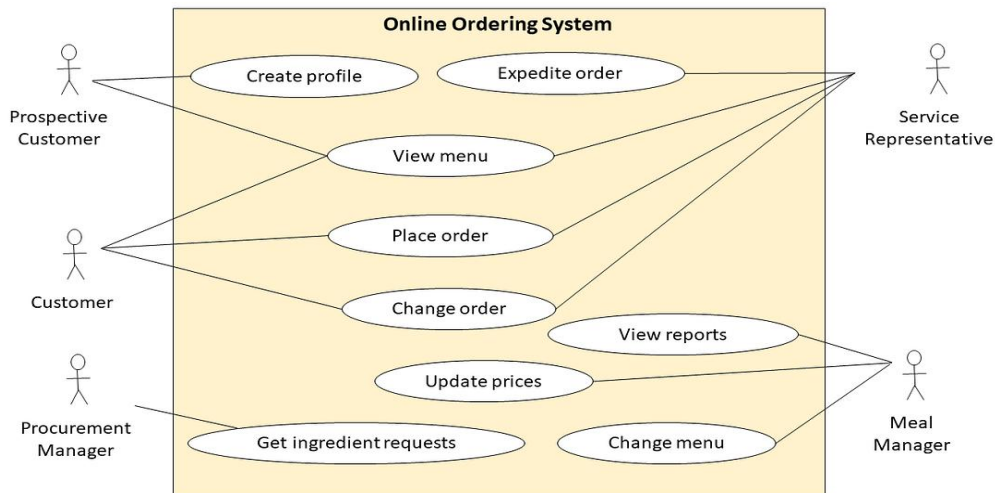
It answers the question:

“What the system should do from the user’s perspective?”

Purpose of Use Case Diagram

- Captures **user requirements**
- Defines **system functionality**
- Provides a **high-level system view**
- Acts as a communication tool between users and developers

SE Paper Solution 2025



Main Components

1. Actor

- Represents a user or external system
- Can be a person, device, or another system

2. Use Case

- Represents a specific function or service
- Shown as an oval

3. System Boundary

- Defines the scope of the system
- Shown as a rectangle

4. Relationships

- Association
- Include
- Extend
- Generalization

Advantages

- Easy to understand
- User-centric
- Helps in requirement validation
- Useful in test case generation

SE Paper Solution 2025

Limitations

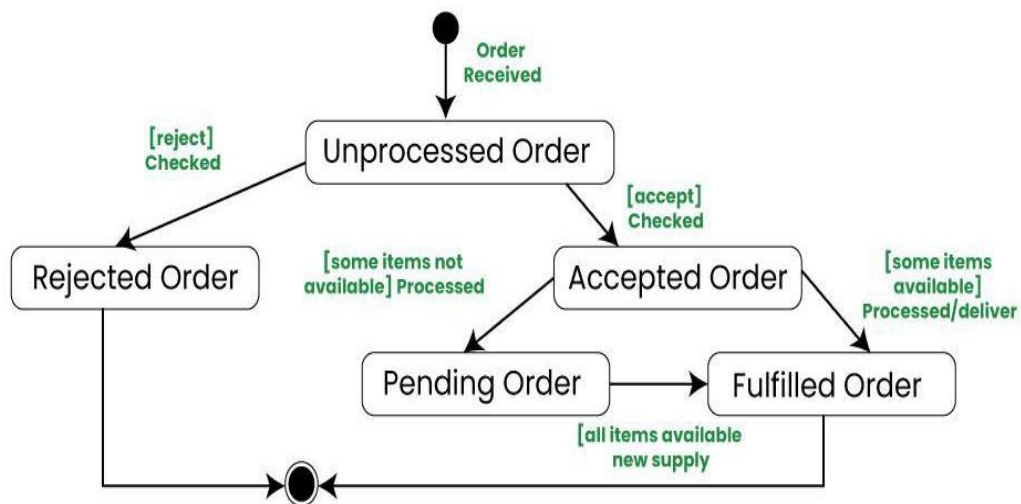
- Does not show internal logic
- No sequence or timing details

2. State Chart Diagram

A **State Chart Diagram** is a **UML behavioral diagram** that represents the **dynamic behavior of an object** by showing **different states and transitions** in response to events.

It answers the question:

“How does the object behave over time?”



Purpose of State Chart Diagram

- Models **object life cycle**
- Shows state changes due to events
- Useful for **real-time and embedded systems**

Main Components

1. State

- Condition in which an object exists
- Represented by a rounded rectangle

2. Event

- Trigger that causes state change

SE Paper Solution 2025

3. Transition

- Movement from one state to another
- Represented by arrows

4. Initial and Final State

- Initial state → solid circle
- Final state → double circle

Advantages

- Clearly shows object behavior
- Helps in system validation
- Useful for control systems

Limitations

- Complex for large systems
- Not suitable for data flow modeling

Q3. Explain Control Flow Diagram (CFD) and Data Flow Diagram (DFD) in detail.

1. Control Flow Diagram (CFD)

A **Control Flow Diagram (CFD)** represents the **flow of control or execution order** in a system.

It shows **decision points, conditions, loops, and sequence of operations**.

CFD answers the question:

“In what order will operations execute?”

Purpose of CFD

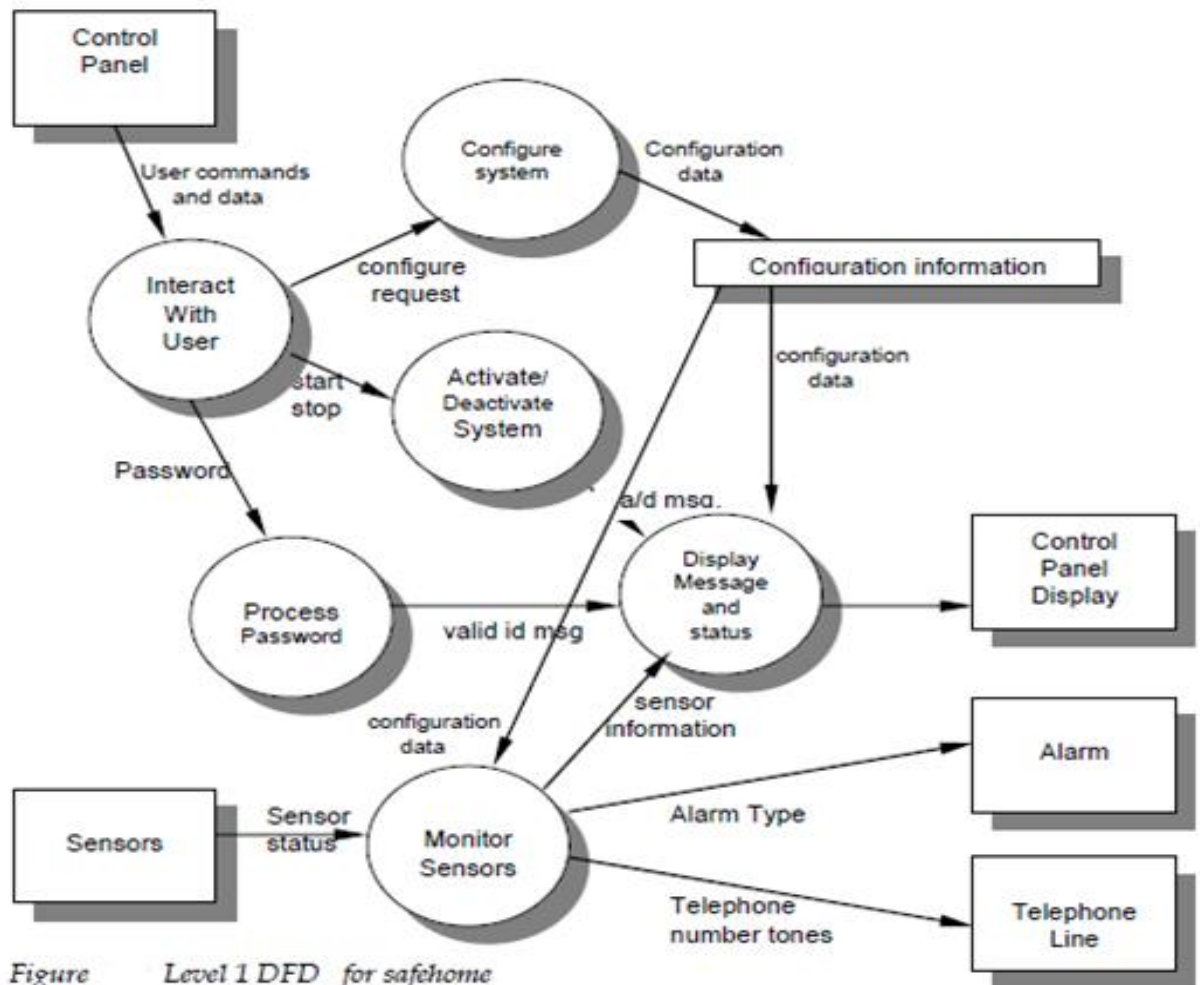
- To represent **decision-making logic**
- To show **conditional branching**
- To model **loops and control structures**
- To analyze **real-time and event-driven systems**

Key Elements of CFD

- Start/End nodes
- Decision nodes (if-else, switch)
- Control paths

SE Paper Solution 2025

- Loops (while, for)



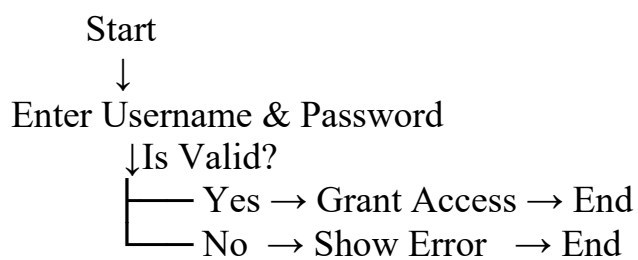
CFD Example (Login System)

User enters username and password.

If valid → access granted

Else → error message

CFD Diagram



SE Paper Solution 2025

Applications of CFD

- Embedded systems
- Real-time system
- Control systems
- Algorithm design

Advantages of CFD

- Clearly shows execution logic
- Helps identify decision complexity
- Useful for control-intensive programs

Limitations of CFD

- Does not show data movement
- Not suitable for data-driven systems
- Becomes complex for large systems

2. Data Flow Diagram (DFD)

A **Data Flow Diagram (DFD)** graphically represents the **flow of data** within a system, showing how data moves from input to processing and finally to output.

DFD answers the question:

“Where does data come from, how is it processed, and where does it go?”

Purpose of DFD

- To analyze **data processing**
- To understand **system functionality**
- To identify **data sources and destinations**
- To improve communication with users

Types of DFD

Level-0 DFD (Context Diagram)

Level-1 DFD

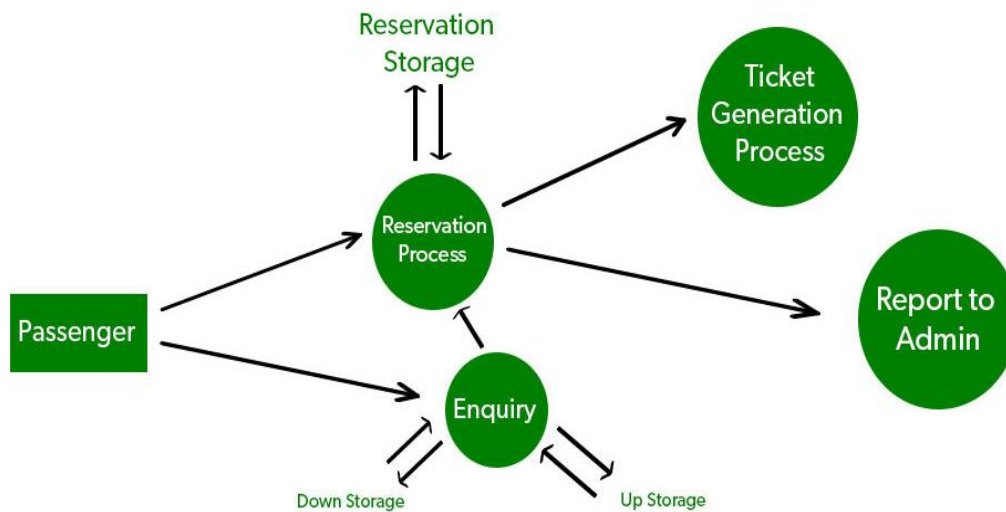
Level-2 DFD (Detailed)

SE Paper Solution 2025

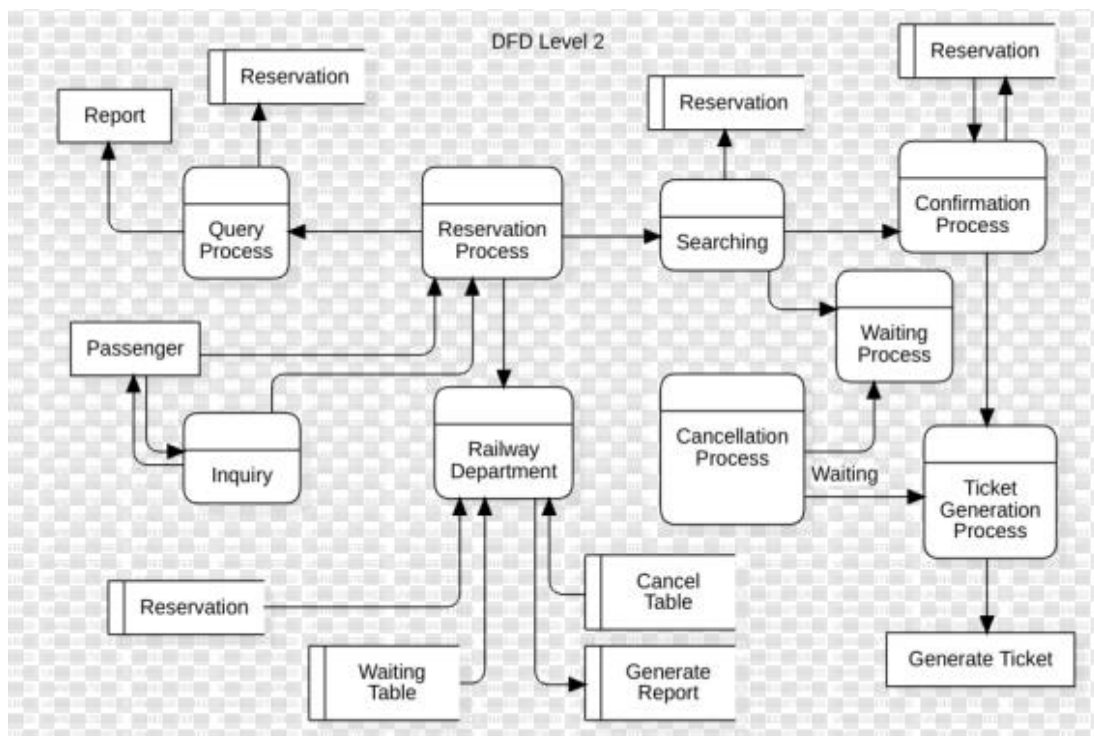
Level-0 DFD



Level-1 DFD



Level-2 DFD



SE Paper Solution 2025

DFD Symbols

Symbol	Meaning
Circle	Process
Arrow	Data Flow
Rectangle	External Entity
Open Rectangle	Data Store

Applications of DFD

- Business systems
- MIS systems
- Database-driven applications
- Information systems

Advantages of DFD

- Easy to understand
- User-friendly
- Clearly shows data movement
- Helps detect missing processes

Limitations of DFD

- Does not show control logic
- No timing information
- Complex systems require many levels

Difference Between CFD and DFD

Aspect	CFD	DFD
Focus	Control logic	Data movement
Shows	Execution order	Data flow
Used for	Real-time systems	Business systems
Decisions	Yes	No
Data storage	No	Yes

SE Paper Solution 2025

Q4. Explain modularity and stepwise refinement with example.

Modularity and Stepwise Refinement

Software design becomes complex when a system grows large. To manage this complexity, **modularity** and **stepwise refinement** are used as core design principles.

1. Modularity

Modularity is a software design technique in which a large system is divided into **smaller, independent modules**, each performing a specific function.

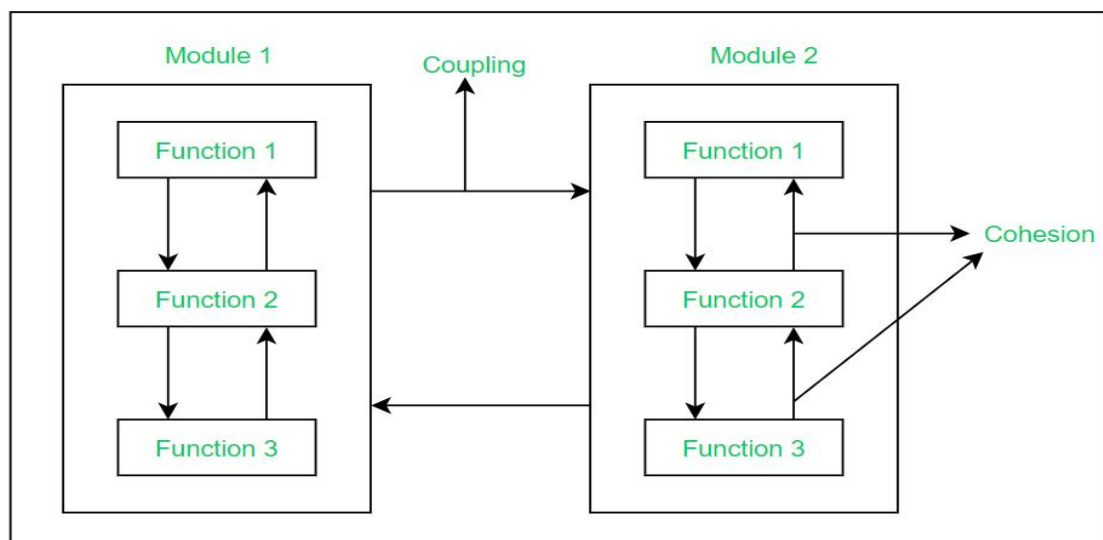
Each module has a **well-defined responsibility** ,Can be developed, tested, and maintained independently

Objectives of Modularity

- Reduce system complexity
- Improve readability and understanding
- Simplify maintenance and debugging
- Enable parallel development

Characteristics of Good Modular Design

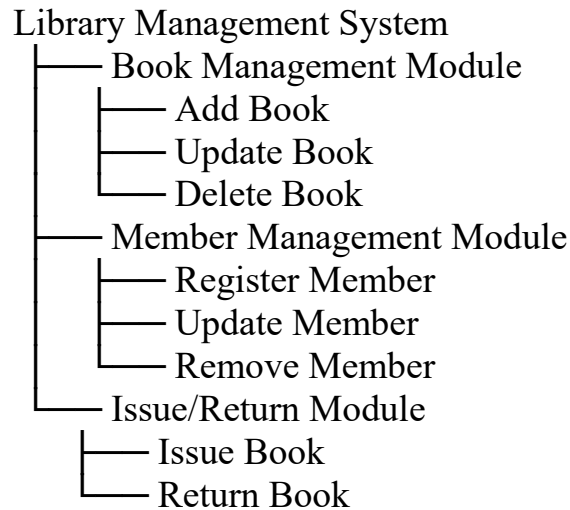
1. **High cohesion** within modules
2. **Low coupling** between modules
3. Clear interfaces
4. Functional independence



SE Paper Solution 2025

Example of Modularity (Library Management System)

System Breakdown



Each module performs a **single, well-defined task**, making the system easy to modify and extend.

Advantages of Modularity

- Easier testing and debugging
- Better maintainability
- Increased reusability
- Reduced development time
- Improved reliability

Limitations of Modularity

- Poor design may increase coupling
- Requires careful planning

2. Stepwise Refinement

Stepwise refinement is a **top-down design approach** in which a complex problem is gradually broken down into **smaller and more detailed sub-problems** until each can be easily implemented.

It focuses on **how to solve the problem**, step by step.

SE Paper Solution 2025

Purpose of Stepwise Refinement

- Simplify complex logic
- Improve clarity of design
- Ensure completeness of solution
- Support structured programming

Example of Stepwise Refinement (ATM Withdrawal)

Level 1 – High-Level Task

Process ATM Withdrawal

Level 2 – Sub-Tasks

Process ATM Withdrawal

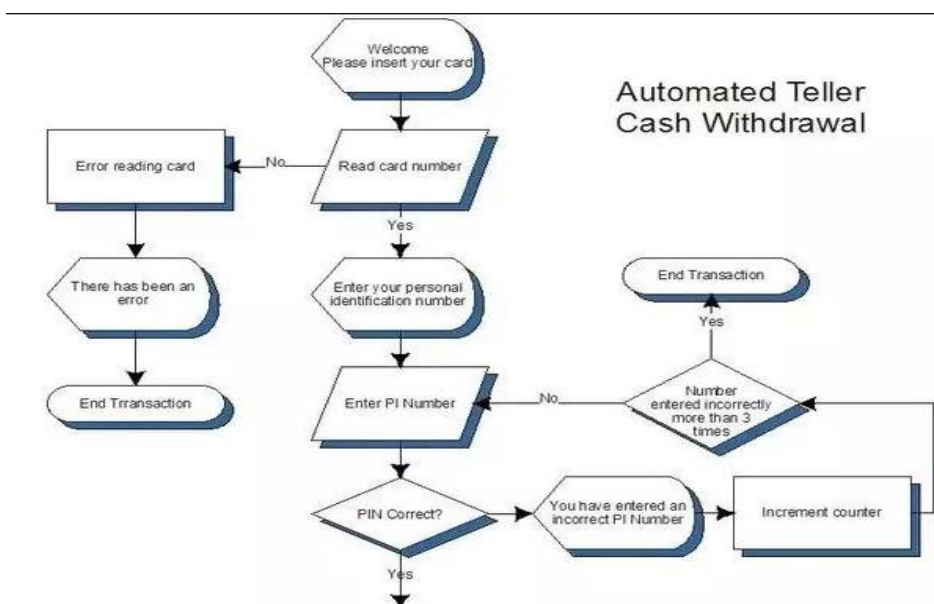
- Authenticate User
- Check Balance
- Dispense Cash
- Update Account

Level 3 – Detailed Refinement

Authenticate User

- Read Card
- Enter PIN
- Verify PIN

Diagram for Stepwise Refinement



SE Paper Solution 2025

Advantages of Stepwise Refinement

- Reduces design errors
- Makes coding easier
- Improves logical structure
- Helps in early error detection

Difference Between Modularity and Stepwise Refinement

Aspect	Modularity	Stepwise Refinement
Focus	System structure	Problem solving
Approach	Divide system into modules	Break logic step-by-step
Level	Structural design	Algorithmic design

Modularity helps manage system complexity by dividing it into independent parts

Stepwise refinement helps in developing solutions in a logical and systematic manners

Together, they form the foundation of **good software design**

Q5. Write Short Notes on following topics:

- (A) System specification
- (B) Software prototyping
- (C) Incremental Model
- (D) V model

(A) System Specification

System Specification is a detailed description of **what a system should do**, including its **functional requirements, non-functional requirements, constraints, and interfaces**.

It acts as a **formal agreement** between the client and developer.

Contents of System Specification

Functional requirements: what the system should do.

SE Paper Solution 2025

User login and authentication, Data processing, Report generation

Non-functional requirements: Define how well the system performs.
Performance (response time < 2 sec), Security, Reliability, Usability

System constraints: Define **limitations or restrictions** on the system.
Hardware limitations, Budget constraints, Programming language restrictions, Legal requirement

Input/output specifications: Defines data entered into and produced by the system.

Input Examples:User details, Transaction data

Output Examples: Reports, Confirmation messages

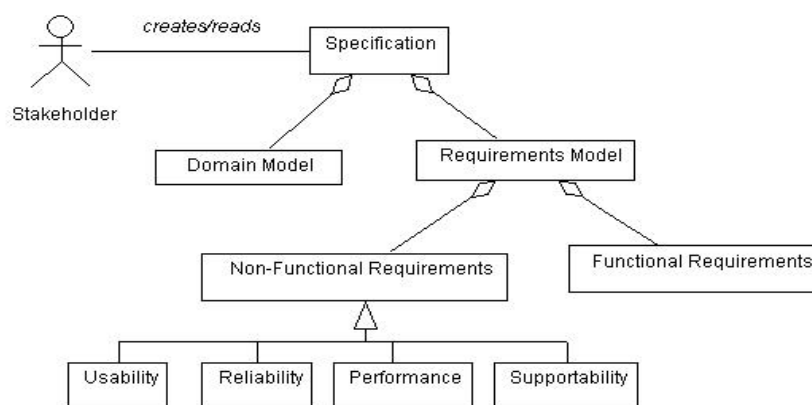
User and system interfaces : Define interaction between users and system or between systems.

User Interface: Screens, forms, menus

System Interface: Interaction with other software/hardware

Purpose

- Provides a clear understanding of the system
- Serves as a base for **design and development**
- Helps in validation and testing



Advantages

- Reduces ambiguity
- Improves communication
- Ensures correctness

SE Paper Solution 2025

(B) Software Prototyping

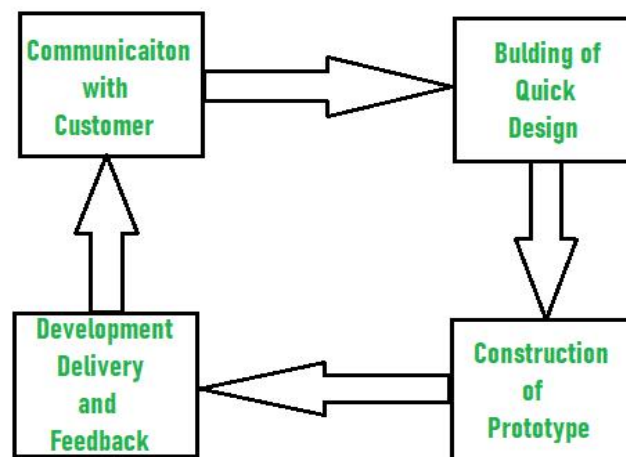
Software Prototyping is a development approach in which a **working model (prototype)** of the system is built to understand and clarify user requirements.

Types of Prototyping

1. Throwaway Prototyping
2. Evolutionary Prototyping

Steps in Prototyping

1. Gather basic requirements
2. Quick design
3. Build prototype
4. User evaluation
5. Refine prototype



Advantages

- Better requirement understanding
- Early user involvement
- Reduces requirement errors

Limitations

- Can increase cost
- Users may expect final system early

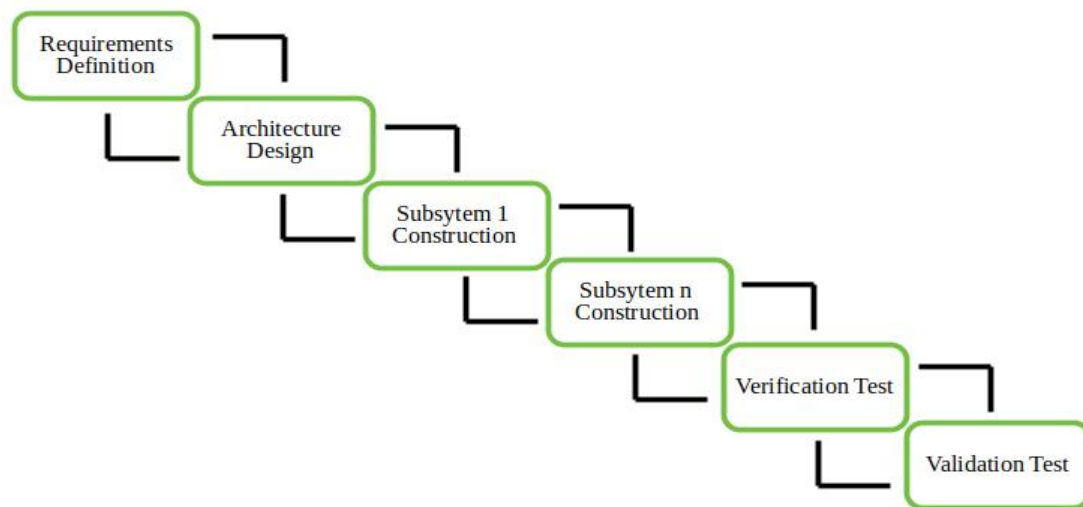
SE Paper Solution 2025

(C) Incremental Model

The **Incremental Model** develops software in **small, manageable increments**, where each increment delivers **working functionality**.

Working

- System is divided into modules
- Each increment goes through SDLC phases
- Final system is a combination of all increments



Advantages

- Early delivery of software
- Easy to handle changes
- Reduced risk

Disadvantages

- Needs good planning
- Not suitable if system is highly interdependent

Applications

- Business applications
- Web applications

SE Paper Solution 2025

(D) V-Model

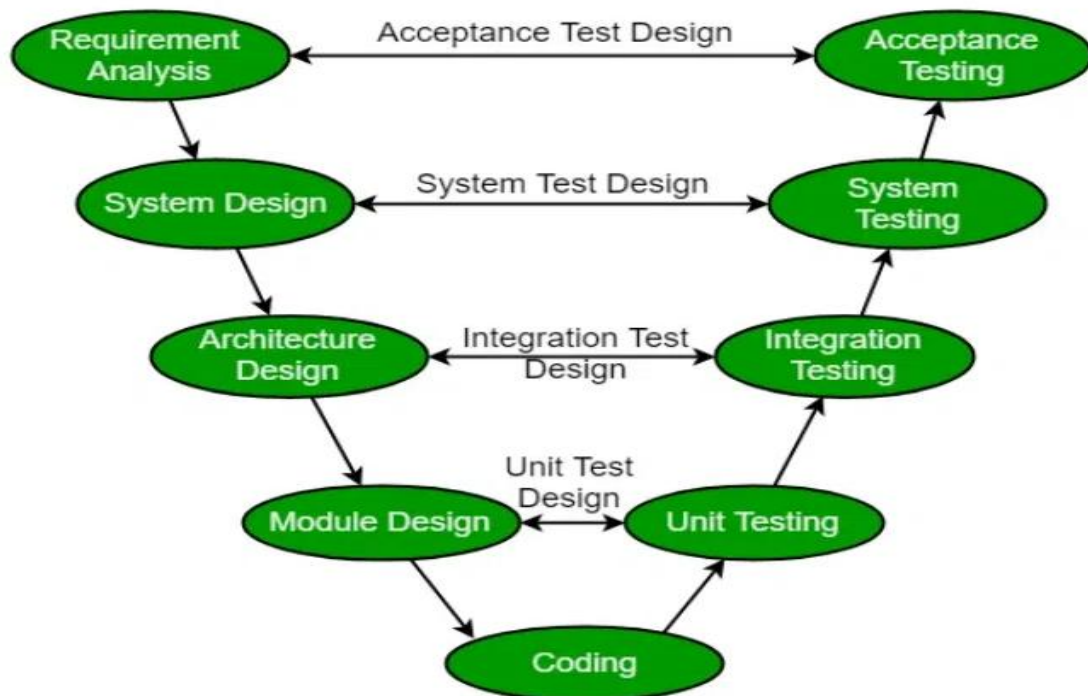
The V-Model (Verification and Validation Model) is an extension of the Waterfall model where testing is planned in parallel with development.

Concept

Left side → Development phases

Right side → Corresponding testing phases

V-Model Diagram



Advantages

- Early test planning
- High quality assurance
- Simple and structured

Limitations

- Rigid model
- Not suitable for changing requirements